1. 전체 시스템에서 백엔드 역할

[Backend]

|

├── ① 고령자 정보 관리

|

├── ② ML 위험도 예측 요청

| ↓

| risk_snapshot 저장

|

├── ③ LLM + RAG 대응계획 요청

| ↓

| intervention_plan 저장

|

├── ④ AI 전화 작업 생성

| ↓

| contact_job 저장

|

├── ⑤ AI 전화 결과 수신

| ↓

| 통화 내용 / STT 결과

|

├── ⑥ LLM + RAG 통화 내용 분석

| ↓

| call_observation 저장

|

└── ⑦ 후속 지원 필요 여부 판단

↓

support_task 생성

2. 고령자 관리 API

가장 기본적으로 elderly_profile을 관리하는 API가 필요해.

구현 내용

고령자 등록

고령자 조회

고령자 수정

지역별 고령자 조회

연락 동의 여부 확인

예를 들어:

POST /api/elderly

GET /api/elderly/{id}

PUT /api/elderly/{id}

GET /api/elderly?regionCode=45113

특히 자동 전화를 사용하는 시스템이기 때문에 consent_status가 동의 상태인 대상에게만 전화가 나가도록 백엔드에서 검증하는 게 중요해.

3. ML 위험도 예측 연동

위험도 예측 모델 자체는 ML 담당 영역이고, 백엔드는 ML 모델을 호출하고 결과를 저장하는 역할을 담당하면 돼.

예를 들어 ML 서버가:

지역 코드

날씨

기온

습도

인구 데이터

과거 위험 데이터

등을 받아서

risk_score = 0.87

risk_level = HIGH

를 반환한다고 하면 백엔드에서 결과를 risk_snapshot에 저장한다.

백엔드 구현

POST /api/risks/predict

처리 과정:

Backend

↓

ML 서버 호출

↓

risk_score / risk_level 수신

↓

risk_snapshot 저장

그리고 특정 시간마다 자동으로 위험도를 계산해야 한다면 스케줄러도 필요해.

예:

매시간

↓

위험도 예측

↓

risk_snapshot 저장

↓

위험 지역 확인

4. LLM + RAG 대응 계획 생성

위험도가 높게 나온 이후에는

"이 지역의 고령자에게 어떤 대응을 해야 하는가?"

를 결정해야 해.

이 부분은 LLM + RAG가 담당하고 백엔드는 필요한 데이터를 전달하고 결과를 저장하는 역할을 하면 된다.

예를 들어 백엔드가:

고령자 정보

+

현재 위험도

+

대피소 정보

를 LLM/RAG 서버에 전달한다.

그러면 LLM이:

```
{
   "guidance": "...",
   "questions": [
      "...",
      "..."
   ],
   "shelter": "덕진구 임시대피소"
}
```

같은 결과를 반환한다.

백엔드는 이 결과를 intervention_plan에 저장한다.

POST /api/plans/generate

처리 흐름

elderly_profile

+

risk_snapshot

+

shelter

↓

Backend

↓

LLM + RAG

↓

guidance

questions

evidence

↓

intervention_plan

여기서 evidence_chunk_ids_json을 저장하는 이유도 중요해.

나중에

"왜 AI가 이 사람에게 이런 안내를 했지?"

라고 물어봤을 때 RAG에서 어떤 문서를 참고했는지 확인할 수 있어.

5. AI 전화 및 음성 안내

이 부분은 백엔드에서 AI 전화 서비스 API와 연동하는 역할을 담당하면 돼.

예를 들어 외부 AI Voice 서비스가 있다고 하면:

Backend

↓

전화 대상 확인

↓

전화 작업 생성

↓

AI Voice API 호출

↓

전화 발신

DB에서는 contact_job이 이 역할을 담당해.

예시

POST /api/contacts/jobs

전화 작업을 만들면:

contact_job

---

id = 100

elderly_id = 1

intervention_plan_id = 10

status = SCHEDULED

scheduled_at = ...

attempt_count = 0

그리고 예약 시간이 되면 백엔드가 AI 전화 API를 호출한다.

6. AI 음성 안내에 대응 계획 전달

AI 전화가 단순히

"안녕하세요."

만 하는 게 아니라 intervention_plan에서 생성된 질문과 안내 내용을 활용해야 해.

예를 들어:

guidance_json

"폭염이 예상됩니다.

현재 집에 계신지 확인해주세요."

그리고

questions_json

1. 현재 집에 계신가요?
2. 혼자 이동할 수 있으신가요?
3. 도움이 필요하신가요?

이 정보를 AI Voice 서비스에 전달한다.

즉 백엔드는 AI가 사용할 대화 컨텍스트를 관리하는 역할을 한다.

7. 통화 결과 수신

AI 전화가 끝나면 외부 전화 서비스에서 결과를 백엔드로 전달해야 한다.

예를 들어:

AI Voice

↓

Webhook

↓

Backend

백엔드에서는 다음과 같은 정보를 받을 수 있어.

통화 상태

통화 ID

통화 종료 시간

음성 파일

STT 결과

그리고 이 결과를 바탕으로 다음 단계인 LLM 분석을 실행한다.

8. 통화 내용 LLM + RAG 분석

여기가 두 번째 AI 분석 부분이야.

STT 결과가:

"대피소에 가고 싶은데 다리가 불편해서 혼자 가기는 어려워요."

라고 들어왔다고 해보자.

백엔드는 이 텍스트를 LLM + RAG에 전달한다.

LLM이:

```
{
   "shelter_intent": "YES",
   "can_move_alone": "NO",
   "help_needed": "YES",
   "symptom_mentioned": "YES",
   "summary": "대피소 이동 의사는 있으나 혼자 이동하기 어려움",
   "confidence": 0.94
}
```

형태로 결과를 반환하도록 만들 수 있다.

백엔드는 이 결과를 call_observation에 저장한다.

STT

↓

Backend

↓

LLM + RAG

↓

분석 결과

↓

call_observation

9. 후속 지원 업무 생성

이 부분은 백엔드의 핵심 역할로 잡으면 좋아.

LLM 분석 결과가:

can_move_alone = NO

help_needed = YES

라면 백엔드에서 자동으로:

support_task

를 생성한다.

예를 들어:

task_type = SHELTER_ESCORT

priority = HIGH

status = TODO

reason = "혼자 대피소 이동이 어려움"

due_at = ...

즉,

AI가 판단

↓

Backend가 업무로 변환

↓

담당자에게 전달

하는 구조야.

10. 백엔드에서 반드시 필요한 스케줄러

이 시스템에서는 스케줄러가 상당히 중요해.

예를 들어:

위험도 예측

매시간

→ ML 호출

→ risk_snapshot 저장

대응 계획

위험도가 HIGH

→ 대상 고령자 조회

→ intervention_plan 생성

전화

scheduled_at 도달

→ contact_job 조회

→ AI 전화 API 호출

재전화

전화 미응답

→ attempt_count 증가

→ 다음 전화 예약

이런 작업을 자동으로 처리해야 해.

Spring을 사용한다면 MVP에서는 @Scheduled로 시작해도 되고, 작업량이 많아지면 Redis Queue나 메시지 브로커 등을 고려할 수 있어.

11. 백엔드에서 구현할 API 정리

MVP 기준으로는 이 정도 API부터 만들면 충분해.

기능	API	역할
고령자 등록	POST /api/elderly	고령자 등록
고령자 조회	GET /api/elderly/{id}	고령자 정보 조회
지역별 조회	GET /api/elderly?regionCode=	위험 지역 대상자 조회
위험도 예측	POST /api/risks/predict	ML 서버 호출
위험도 조회	GET /api/risks/{id}	위험도 결과 조회
대응계획 생성	POST /api/plans/generate	LLM + RAG 호출
대응계획 조회	GET /api/plans/{id}	대응계획 확인

전화 작업 생성 POST /api/contacts/jobs 전화 예약

전화 작업 조회 GET /api/contacts/jobs 전화 작업 확인

전화 시작 POST /api/contacts/jobs/{id}/call/start AI 전화 시작

전화 결과 POST /api/contacts/jobs/{id}/call/result AI 전화 결과 수신

통화 결과 조회 GET /api/observations/{id} 통화 분석 결과 조회

지원 업무 생성 POST /api/support/tasks 후속 지원 생성

지원 업무 조회 GET /api/support/tasks 지원 업무 조회

12. 백엔드와 AI의 역할을 명확하게 나누면

이게 해커톤에서 특히 중요해.

영역 담당

고령자/대피소 데이터 관리 Backend

위험도 계산 ML

위험도 결과 저장 Backend

RAG 검색 LLM/RAG

대응 문장/질문 생성 LLM

대응 계획 저장 Backend

전화 발신 AI Voice 서비스

TTS AI Voice

STT AI Voice

통화 내용 분석 LLM + RAG

통화 분석 결과 저장 Backend

지원 업무 생성 Backend

스케줄링 Backend

API 제공 Backend

인증/권한 Backend

로그/에러 처리 Backend

13. 해커톤 MVP에서 백엔드가 우선 구현할 것

시간이 부족하다면 전부 완벽하게 만들 필요는 없고, 아래 순서로 구현하는 걸 추천해.

1순위 — DB + Entity

elderly_profile

risk_snapshot

shelter

intervention_plan

contact_job

call_observation

support_task

Spring JPA Entity 및 Repository 구현.

2순위 — ML 연동

Backend

→ ML API

→ risk_snapshot 저장

3순위 — LLM/RAG 연동

Backend

→ LLM/RAG API

→ intervention_plan 저장

4순위 — AI 전화 연동

contact_job

→ AI Voice API

→ 전화 발신

5순위 — 통화 결과 처리

AI Voice

→ Backend Webhook

→ STT

→ LLM 분석

→ call_observation 저장

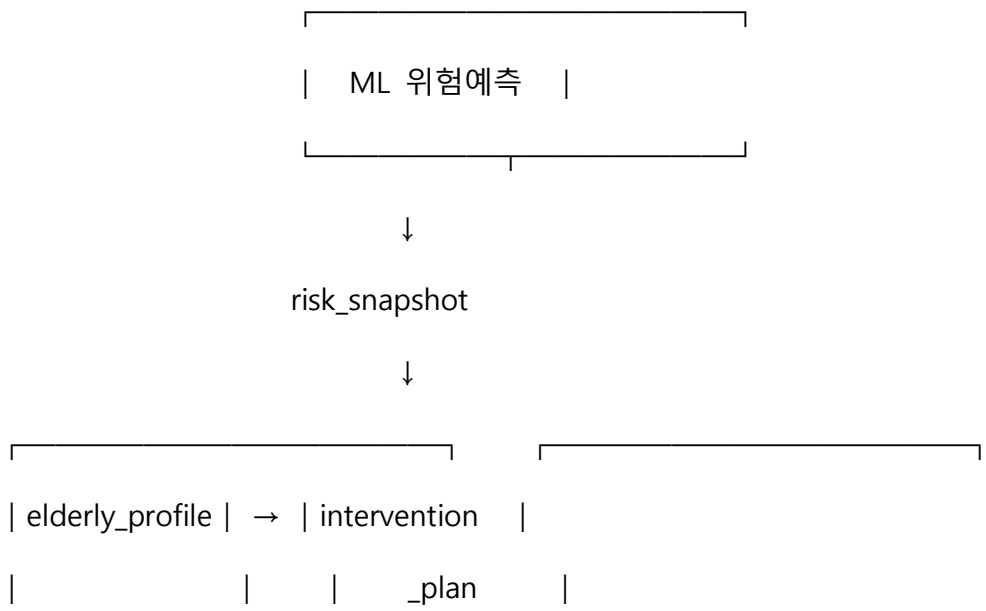
6순위 — 지원 업무 자동 생성

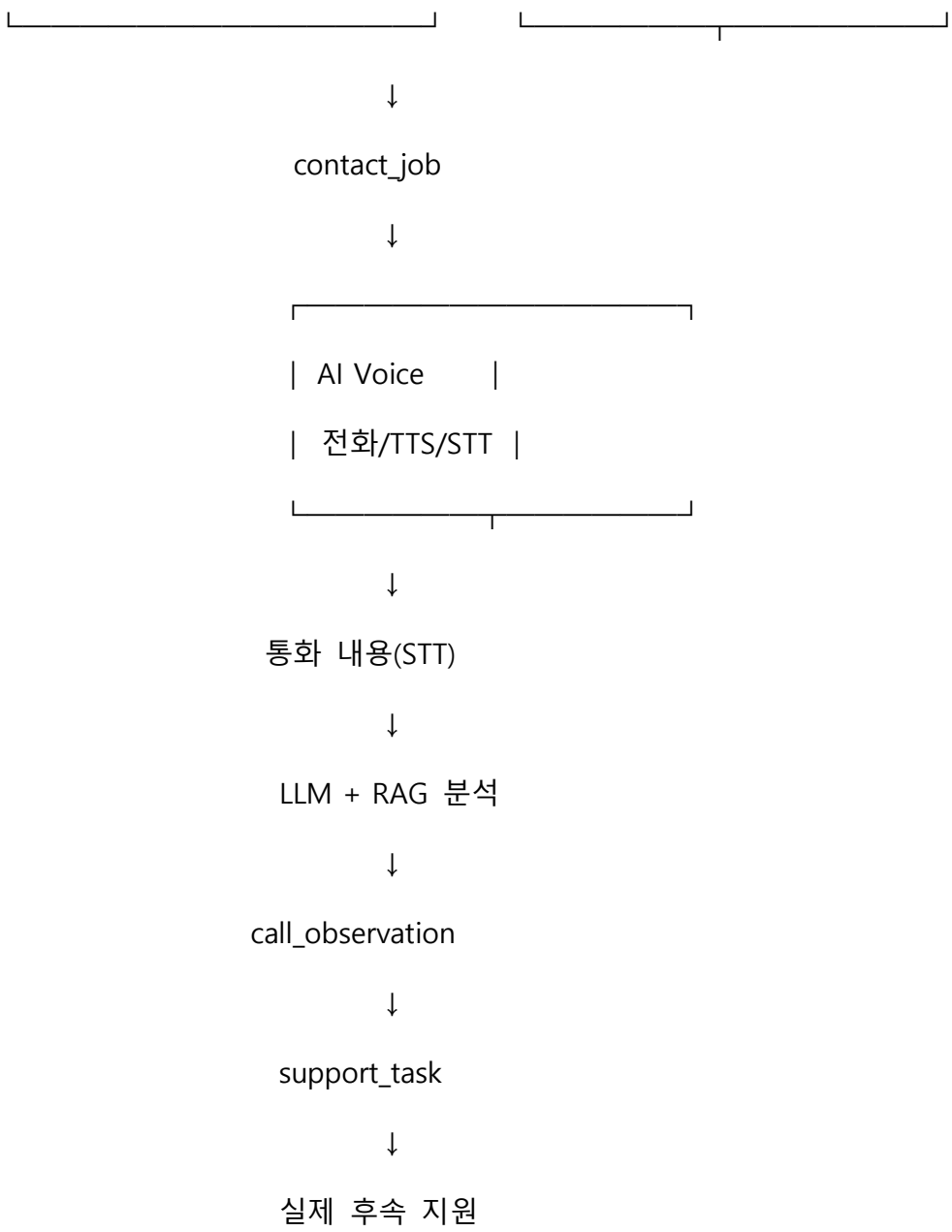
call_observation

→ 조건 확인

→ support_task 생성

최종적으로 백엔드가 만들어야 하는 핵심 그림





결국 백엔드의 핵심은 AI를 직접 만드는 게 아니라 데이터 → AI 호출 → 결과 저장 → 다음 작업 생성을 안정적으로 연결하는 오케스트레이션 역할이라고 보면 돼.

해커톤에서는 특히 ① DB/JPA → ② AI API 연동 → ③ 스케줄러 → ④ Webhook → ⑤ 상태 관리 이 다섯 가지를 백엔드 핵심 구현 범위로 잡으면 역할 분담하기 좋다.